

- Architectural Change Document: Next-Generation Invoice Processing Pipeline
 - Executive Summary
 - 1. Previous Architecture: Azure prebuilt-invoice
 - 2. New Architecture: Hybrid prebuilt-layout + LLM
 - 3. Architectural Comparison
 - 4. Why Accuracy Improved Drastically
 - A. Separation of Concerns (Vision vs. Comprehension)
 - B. The Power of Markdown
 - C. Contextual and Semantic Resolution
 - D. Prompt-Level Constraints and Self-Correction
 - 5. Conclusion

Architectural Change Document: Next-Generation Invoice Processing Pipeline

To: Engineering Management **From:** Engineering Team **Date:** May 15, 2026 **Subject:** Architectural Transition from Azure prebuilt-invoice to Hybrid prebuilt-layout + LLM Pipeline

Executive Summary

This document proposes and details the architectural transition of our document intelligence pipeline from using the rigid, template-based Azure Document Intelligence prebuilt-invoice model to a highly flexible, hybrid architecture utilizing Azure prebuilt-layout combined with a Large Language Model (LLM) for semantic extraction. This transition has resulted in a **drastic improvement in data extraction accuracy**, particularly for region-specific documents (such as Indian GST invoices) and highly variable, non-standard layouts.

1. Previous Architecture: Azure prebuilt-invoice

In the legacy architecture, the system relied entirely on Azure's dedicated model for invoices.

Workflow:

1. **Input:** Raw invoice image/PDF.
2. **Processing:** Azure **prebuilt-invoice** model processes the document.
3. **Extraction:** The model attempts to map the visual data to its internal, pre-defined schema (e.g., **VendorName**, **InvoiceTotal**).
4. **Output:** A structured JSON object mapped directly to our application models.

Limitations:

- **Rigid Schema:** The model was trained heavily on western/standardized invoice formats. It struggled significantly with Indian-specific fields like **GSTIN**, **HSN/SAC** codes, and complex tax breakdowns (CGST/SGST/IGST).
- **Template Dependency:** When encountering novel, nested, or poorly formatted tables, the model often failed to associate line items with their correct totals.
- **Lack of Semantic Reasoning:** The system could not use context to resolve ambiguities (e.g., distinguishing between a billing address and a shipping address when labels were implicit).

2. New Architecture: Hybrid **prebuilt-layout** + LLM

The new architecture decouples the Optical Character Recognition (OCR)/spatial extraction from the semantic reasoning/data mapping phase.

Workflow:

1. **Input:** Raw invoice image/PDF.
2. **Spatial Processing:** Azure **prebuilt-layout** processes the document, focusing solely on structure (tables, paragraphs, headers).
3. **Markdown Generation:** Azure outputs a highly accurate **Markdown representation** of the document, preserving spatial and tabular relationships.
4. **Semantic Reasoning:** The Markdown text is injected into an advanced LLM via the Groq API (e.g., **openai/gpt-oss-120b**).

- 5. **Prompt-Driven Extraction:** The LLM is given a precise system prompt, rules (e.g., "Total Amount MUST equal Taxable Amount + GST Amount", "GSTIN must be 15 chars"), and a target JSON schema.
- 6. **Output:** The LLM returns a strictly validated JSON object mapped to our application schema.

3. Architectural Comparison

Feature	Legacy: prebuilt-invoice	New: prebuilt-layout + LLM
Primary Mechanism	End-to-end black-box model.	OCR/Structure mapping -> Semantic LLM parsing.
Data Representation	Key-Value pairs based on internal confidence.	Structured Markdown preserving document flow and tables.
Adaptability	Low. Requires waiting for Microsoft to update the model for new formats or regions.	Extremely High. Changes to extraction logic only require modifying the LLM system prompt.
Custom Fields	Very difficult. Often requires retraining or custom model creation.	Trivial. Simply add the new field definition and rules to the prompt schema.
Validation Logic	Post-processing required in our backend (e.g., checking if totals add up).	Handled inherently by the LLM via prompt constraints before output generation.

4. Why Accuracy Improved Drastically

The transition resulted in a massive leap in accuracy for the following core reasons:

A. Separation of Concerns (Vision vs. Comprehension)

The previous model tried to do both OCR and semantic mapping simultaneously. By switching to `prebuilt-layout`, we let Azure do what it does best: flawlessly read text and recognize table grids. We then let the LLM do what it does best: understand language, context, and intent.

B. The Power of Markdown

Markdown is the perfect bridge format. `prebuilt-layout` outputs tables as Markdown tables (e.g., `| Item | Qty | Price |`). This format is native to how LLMs are trained, allowing the LLM to easily traverse rows and columns to find line-item details, something standard key-value extractors struggle with when tables span multiple pages.

C. Contextual and Semantic Resolution

Invoices often have implicit data. For example, a GSTIN might just be a 15-character string near the top of the page without a clear "GSTIN:" label. The `prebuilt-invoice` model would often miss this. The LLM, given its semantic understanding of Indian accounting rules and the context of the surrounding text, easily identifies the string as a valid GSTIN.

D. Prompt-Level Constraints and Self-Correction

We embedded business logic directly into the LLM prompt:

- *"Total Amount MUST equal Taxable Amount + GST Amount."*
- *"GSTIN must be exactly 15 alphanumeric characters."*

Because the LLM "thinks" before generating the final JSON, it can use these rules to self-correct edge cases (e.g., resolving a poorly scanned '8' as a 'B' if the math doesn't align).

5. Conclusion

Transitioning to the hybrid **prebuilt-layout** + LLM architecture represents a shift from a rigid, vendor-locked extraction method to an agile, intelligent, and schema-agnostic data pipeline. The drastic improvement in accuracy directly reduces manual human review time and stabilizes our accounting data flows, making the system truly production-ready.